Logic Tensor Networks

Neuro-Symbolic Artificial Intelligence

Francesca Cairoli

University of Trieste, 2026/27

Introduction

- Logic Tensor Networks (LTN) is a NSR framework based on fuzzy differentiable logic that supports tasks based on manipulating **data** and **knowledge**.

- It has been shown to effectively tackle many tasks that are central to intelligent systems:
  - multi-label classification,
  - relational learning,
  - data clustering,
  - semi-supervised learning,
  - regression,
  - embedding learning, and
  - query answering under uncertainty.

Material

Logic Tensor Networks (LTN)

- Git Repository (PyTorch): https://github.com/tommasocarraro/LTNtorch.git
- Paper: https://www.sciencedirect.com/science/article/pii/S0004370221002009

Goal

Learning of a neural network using the **satisfaction** of a first order logic knowledge base **as an objective**.

LTN uses logical reasoning on the knowledge base to guide the learning of a deep neural network

Running Example

Classification with Nearest Neighbor

The domain is the square $[0,4] \times [0,4]$. We have some points in this domain, and we need to infer their classes.

In particular, we know the classes of only two examples. One example is of class A and the other of class B.

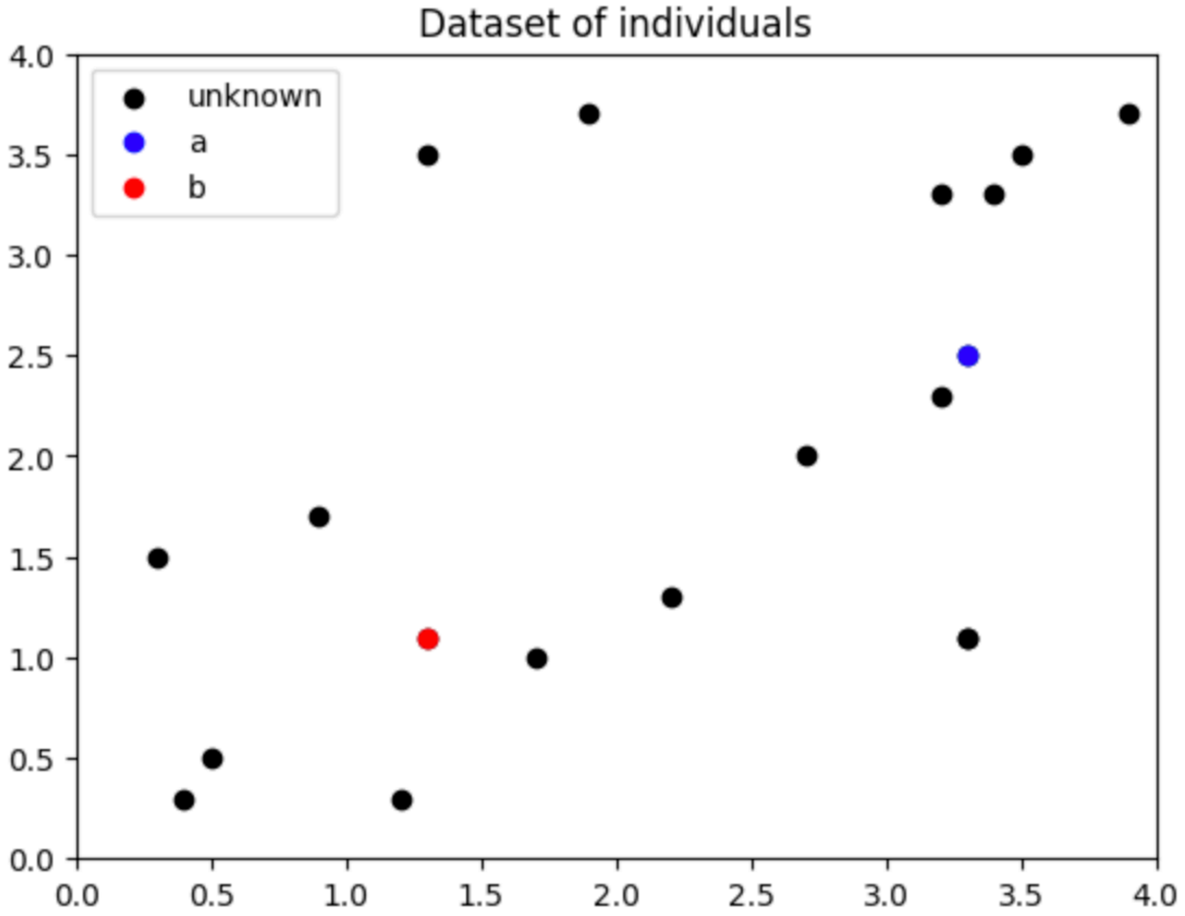
The rest of the points are not labelled, but there are two assumptions:

- A and B are mutually exclusive,
- any two close points should share the same label.

Running Example

Classification with Nearest Neighbour

In the following, we plot a dataset made of 19 points distinguishing between classified and not classified examples.



Running Example

Definition of knowledge base

First, we need to define a *membership predicate* $C(x, l)$, where x is an individual (point) and l is a one-hot label denoting one of the two classes (10 for class A and 01 for class B).

C is approximated by a simple MLP (Multi-Layer Perceptron). The last layer, that computes probabilities per class, uses a softmax activation, ensuring that the classes are mutually-exclusive.

Running Example

Definition of knowledge base

$$
\begin{gathered}
C(a, l_a) \\
C(b, l_b) \\
\forall x_1, x_2, l \left(\mathrm{Sim}(x_1, x_2) \rightarrow \left(C(x_1, l) \leftrightarrow C(x_2, l) \right) \right)
\end{gathered}
$$

where:

- a and b the two individuals already classified;
- x_1 and x_2 are variables ranging over all individuals;
- l_a and l_b are the one-hot labels for A and B;
- l is a variable ranging over the labels;
- Sim is a predicate measuring similarity between two points and defined as $\mathcal{G}(\mathrm{Sim}) : \vec{u}, \vec{v} \mapsto \exp(-\|\vec{u} - \vec{v}\|^2)$.

Running Example

Learning objective

The objective is to learn the predicate C in in such a way to maximize the satisfaction of $\mathscr{K}$. If θ denotes the set of trainable parameters, the task is:

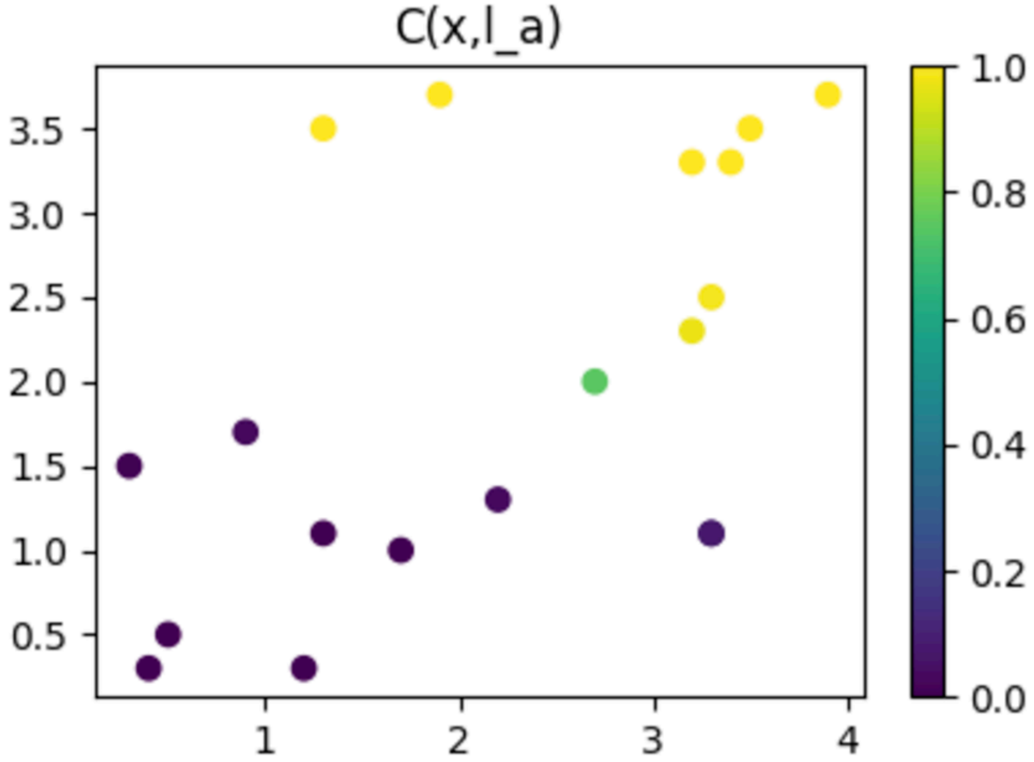
$$\theta^* = \arg\max_{\theta \in \Theta} SatAgg_{\phi \in \mathscr{K}} \mathscr{G}_\theta(\phi).$$

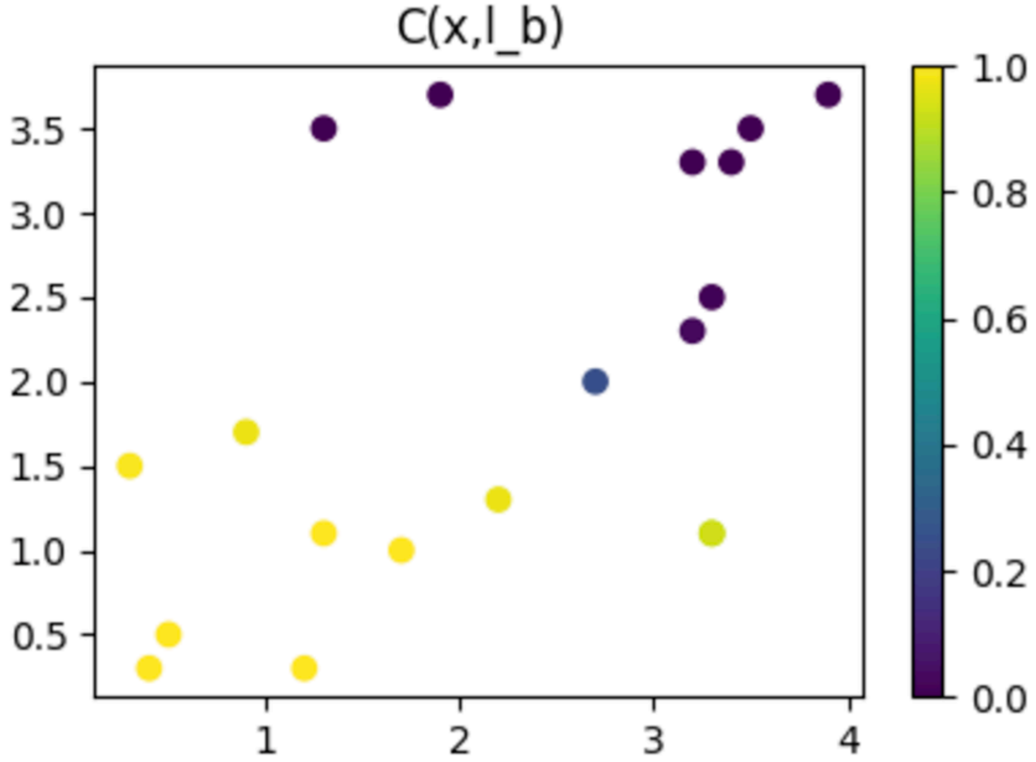
where $SatAgg$ is an operator that aggregates the truth values of the formulas in $\mathscr{K}$ (if there are more than one formula).

Running Example

Learned Predicate

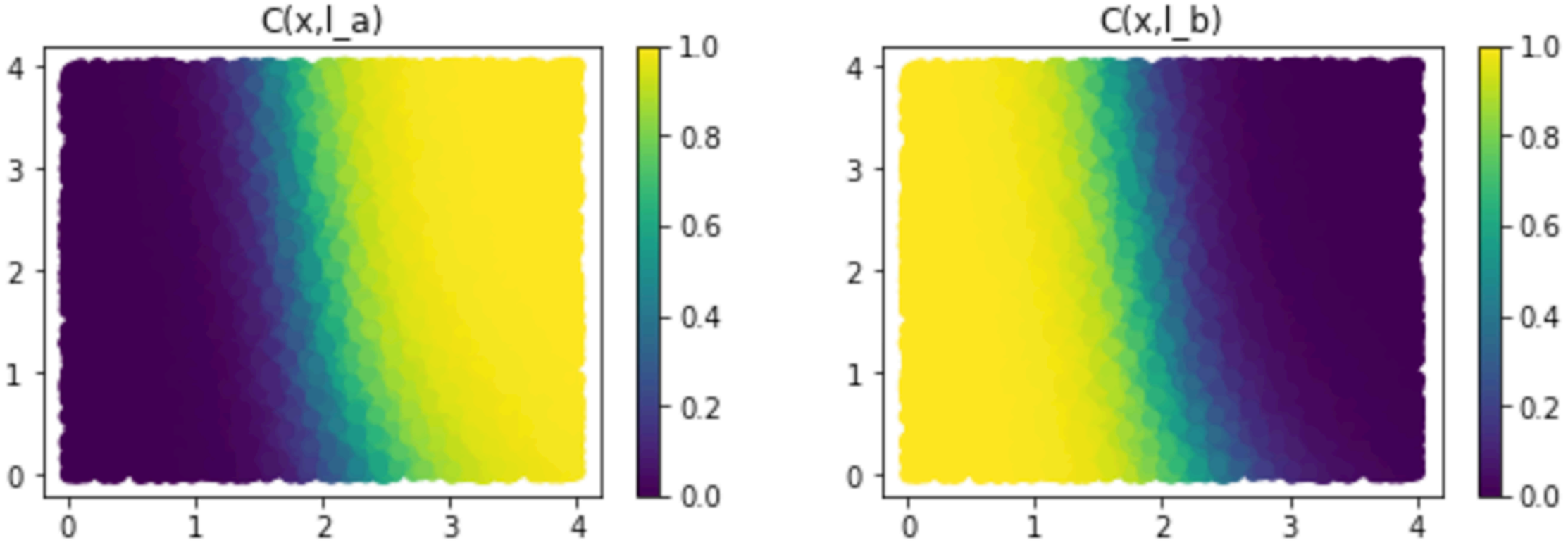
```
Epoch 0: Sat Level 0.532
Epoch 200: Sat Level 0.649
Epoch 400: Sat Level 0.935
Epoch 600: Sat Level 0.950
Epoch 800: Sat Level 0.954
Epoch 1000: Sat Level 0.955
Epoch 1200: Sat Level 0.955
Epoch 1400: Sat Level 0.955
Epoch 1600: Sat Level 0.955
Epoch 1800: Sat Level 0.955
Training finished at Epoch 1999 with Sat Level 0.955
```





Running Example

Learned Predicate (over 10000 points from only two labeled data)



Motivational Example

Multi-class multi-label classification

Multiple labels can be assigned to each example, which is not always a trivial extension for most ML algorithms. LTN can use background knowledge to relate classes directly to each other, thus becoming a powerful tool in the case of the multi-label problem, where typically the labelled data is scarce.

Ex: Dataset, consisting of 200 examples of 5 morphological measurements of 50 crabs. The task is to classify the crabs according to their colour and sex. There are four labels: **blue, orange, male, and female**. The colour labels are mutually-exclusive, and so are the labels for sex. LTN will be used to specify such information logically.

Multi-class multi-label classification

Predicate $P(x, l)$ denoting the fact that item x is labelled as l.

Axioms:

- $\forall x_{blue} P(x_{blue}, l_{blue})$: all the examples coloured by blue should have label l_{blue};
- $\forall x_{orange} P(x_{orange}, l_{orange})$: all the examples coloured by orange should have label l_{orange};
- $\forall x_{male} P(x_{male}, l_{male})$: all the examples that are males should have label l_{male};
- $\forall x_{female} P(x_{female}, l_{female})$: all the examples that are females should have label l_{female};
- $\forall x \neg(P(x, l_{blue}) \wedge P(x, l_{orange}))$: if an example x is labelled as blue, it cannot be labelled as orange too;
- $\forall x \neg(P(x, l_{male}) \wedge P(x, l_{female}))$: if an example x is labelled as male, it cannot be labelled as female too.

Notice how the last two logical rules represent the mutual exclusion of the labels on colour and sex, respectively. As a result, negative examples are not used explicitly in this specification.

$G(P \mid \theta) : x, l \mapsto l^T \cdot \sigma(MLP_\theta(x))$ where MLP has four output neurons corresponding to as many labels, and $\cdot$ denotes the dot product as a way of selecting an output for $\mathcal{G}(P \mid \theta)$. In fact, multiplying the MLP's output by the one-hot vector l^T gives the probability corresponding to the label denoted by l. By contrast with the previous example, notice the use of a *sigmoid* function instead of a *softmax* function. We need that because labels are not mutually exclusive anymore.

Querying

After the training phase, we can query the learned model to check which constraints the LTN learned.

Example of three queried formulas that were not explicitly part of the knowledge base, over time during learning:

- $\phi_1 : \forall x(P(x, l_{blue}) \implies \neg P(x, l_{orange}));$ (every blue crab cannot be orange and vice-versa)
- $\phi_2 : \forall x(P(x, l_{blue}) \implies P(x, l_{orange}));$ (every blue crab is also orange)
- $\phi_3 : \forall x(P(x, l_{blue}) \implies P(x, l_{male})).$ (every blue crab is male)

ϕ_1 are expected to hold true, whereas ϕ_2 and ϕ_3 are expected to be false.

Question: How do we ground these formulas over the trained model?

Sketch of the Idea

We have:

- a first-order logic knowledge base containing a set of axioms;
- some predicates, functions, or logical constants appearing in these axioms that we want to learn;
- some data available that we can use to learn the parameters of those symbols.

The idea is to use the logical axioms as a loss function for our Logic Tensor Network.

The objective is to find solutions in the hypothesis space that maximally satisfy all the axioms contained in our knowledge base.

Learning in LTN

In LTN, the learnable parameters are contained in the predicates, functions, and possibly learnable logical constants that appear in the logical axioms of the knowledge base.

During the **forward step** of the back-propagation algorithm, LTN computes the truth values of the logical formulas contained in the knowledge base, using the available data to ground (or instantiate) the logical formulas.

At the end of the forward phase, the truth values computed for the formulas are aggregated and used in the loss function.

Our **objective** is to maximize the aggregated truth value, namely maximally satisfy all the axioms.

Learning in LTN

During the **backward step**, the learnable parameters of predicates, functions, and possibly learnable logical constants are changed in such a way to move towards a solution in the hypothesis space which better satisfies all the axioms in the knowledge base.

At the end of the training, the parameters of predicates, functions, and constants will have been updated in such a way that the logical formulas in the knowledge base are maximally satisfied. In particular, the parameters will have been learned by using both data (to ground the formulas) and logical reasoning (at the loss function).

After learning, it is possible to query predicates and functions on new data which was not available during training. Also, it is possible to query the truth values of new formulas which were not included in the knowledge base during training. In addition, if some logical constants have been learned, their parameters can be interpreted as embeddings.

Learning in LTN

As in classical ML setups, objects are represented by points embedded in a feature space. LTN makes also functions and predicates learnable.

Logical combinations of the basic elements can be used during the learning process in order to guide it.

Weights are thus optimized both when learning basic classifier and regressors, but also in a way that additional constraints provided via formulas are satisfied as much as possible for all points in the feature space.

Real Logic language

To make this learning possible, LTN uses a differentiable first-order logic language, called **Real Logic**, which enables the incorporation of data and logic.

Real Logic defines the concept of **grounding** (different from the grounding of logic), which is a mapping from the logical domain (i.e., constants, variables, and logical symbols) to tensors in the real field or operations based on tensors. These operations could be, for instance, mathematical functions or learnable neural networks.

In other words, a grounding, denoted as $\mathcal{G}$, is a function which maps a logical symbol into a real tensor or an operation on tensors.

Real Logic: grounding

The grounding defines also how the logical connectives ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\leftrightarrow$)

and quantifiers ($\forall$, $\exists$) are mapped in the real domain.

Logical connectives are grounded using fuzzy logic semantics, while quantifiers are grounded using fuzzy aggregators.

Real Logic

Real Logic: Introduction

- Domains are interpreted concretely by tensors in the Real field.
- Recall that tensors are algebraic objects that include:
  - Scalars: 0-dimensional,
  - Vectors: 1-dimensional,
  - Matrices: 2-dimensional,
  - as well as higher-dimension structures.
- Grounding $\mathscr{G}$ maps:
  - Terms to tensors of real numbers, and
  - Formulas to real numbers in the interval [0,1].
- As usual, the language allows for logical connectives and quantifiers

Real Logic: Language

Constants:

- Denote individuals from a space of tensors $\bigcup_{n_1,\ldots,n_d} \mathbb{R}^{n_1 \times \cdots \times n_d}$ (aka “tensor of any rank”).

- The individual can be pre-defined (data point) or learnable (embedding).

- **Intuition:** Each dimension corresponds to a feature, and the number corresponds to the value of that feature for that individual.

Variables denote sequences of individuals:

- Sequences represent the possible values that the variable can take.

- They can contain more than one instance of the same value.

Real Logic: Language

Functions:

- Can be any mathematical function (either pre-defined or learnable).
- Examples of functions are distance functions, regressors, etc.

Predicates:

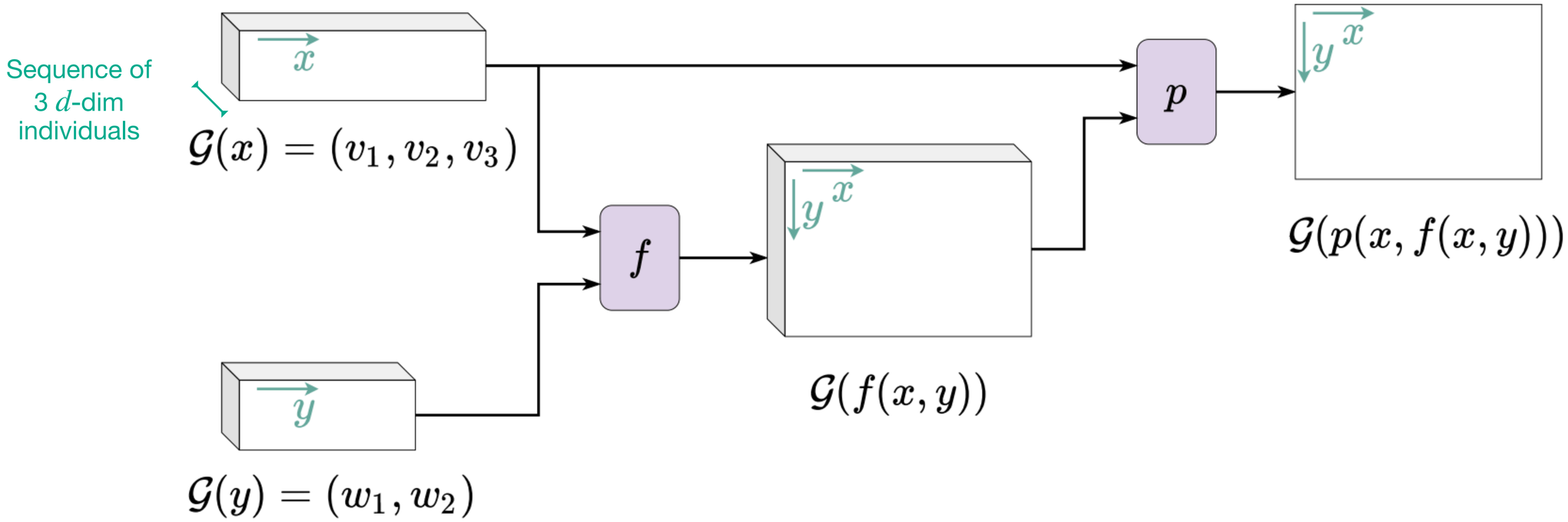
- Represented as mathematical functions that map an n-ary domain of individuals to a real in [0,1], interpreted as a truth degree.
- Examples of predicates: similarity measures, classifiers, etc.

Real Logic: grounding

Let us assume that c is a constant, x is a logical variable, P is a predicate, and f is a logical function:

- $\mathcal{G}(c) = \mathbb{R}^{d_1 \times \cdots \times d_n}$: a logical constant is grounded into a tensor (individual) of **any order** (e.g., $\mathbb{R}^4$ or $\mathbb{R}^{5 \times 4 \times 4}$);
- $\mathcal{G}(x) = \mathbb{R}^{m \times d}$: a logical variable is grounded into a **sequence** of m tensors (individuals) of the same shape d;
- $\mathcal{G}(f \mid \theta) = x \mapsto MLP_\theta(x)$: a logical function is grounded into a (learnable) **mathematical function** which takes as input some tensors (individuals) and returns a tensor. In this definition, θ are the learnable parameters of the function, while MLP_θ is the neural network representing the function, parametrized by θ. Note that the given definition has one input x, however, an LTN function can take multiple inputs;
- $\mathcal{G}(P \mid \theta) = x \mapsto \sigma(MLP_\theta(x))$: a logical formula (atomic or not) is grounded into a mathematical function which takes as input some tensors (individuals) and returns a value in **[0., 1.]**. In this case, the logistic function σ assures the output to be in the range [0., 1.], resulting in a value which can be interpreted as a fuzzy truth value. Note that the given definition has one input x, however, an LTN predicate (or formula) can take multiple inputs.

Real Logic: Grounding of Functions and Predicates



Source: Badreddine, S., Garcez, A. D. A., Serafini, L., & Spranger, M. (2022). Logic tensor networks. Artif. Intelli., 303, 103649.

Real Logic: Language

Connectives are modeled using fuzzy semantics:

- **Conjunction** ($\wedge$): t-norm T
- **Disjunction** ($\vee$): t-conorm S
- **Implication** ($\Rightarrow$): fuzzy implication I
- **Negation** ($\neg$): fuzzy negation N

$FuzzyOp \in \{T, S, I, N\}$ (we will come back to these operators later when discussing differentiability).

Fuzzy logic operators preserve their classical semantics for Boolean inputs $\{0,1\}$.

Real Logic: Common Fuzzy Operators

Fuzzy truth values

$$a, b, c \in [0, 1]$$

Operators

- $\wedge$ Conjunction (T-norm)
- $\vee$ Disjunction (T-conorm)
- $\rightarrow$ Implication

Families of implication $I_S(a, b) = S(N(a), b)$

- **S-implication:** defined from $\vee$ and $\neg$ (e.g., Reichenbach)
- **R-implication:** residuum of a T-norm (e.g., Gödel)

$$I_R(a, b) = \sup\{c \in [0,1] \mid T(a, c) \leq b\}$$

Fuzzy conjunction, disjunction and implication

Logic	$a \wedge b$ (T)	$a \vee b$ (S)	$a \rightarrow_R c$ (R-implication)	$a \rightarrow_S c$ (S-implication)
Gödel	$\min(a, b)$	$\max(a, b)$	$\begin{cases} 1, & \text{if } a \leqslant c \\ c, & \text{otherwise} \end{cases}$	$\max(1 - a, c)$
Product (Goguen)	$a \cdot b$	$a + b - a \cdot b$	$\begin{cases} 1, & \text{if } a \leqslant c \\ \frac{c}{a}, & \text{otherwise} \end{cases}$	$1 - a + a \cdot c$
Łukasiewicz	$\max(a + b - 1, 0)$	$\min(a + b, 1)$	$\min(1 - a + c, 1)$	$\min(1 - a + c, 1)$

Common fuzzy implications

Name	$I(x, y)$	Type
Kleene–Dienes I_{KD}	$\max(1 - x, y)$	S
Gödel I_G	$\begin{cases} 1, & x \leqslant y \\ y, & \text{otherwise} \end{cases}$	R
Reichenbach I_R	$1 - x + xy$	S
Goguen I_P	$\begin{cases} 1, & x \leqslant y \\ y/x, & \text{otherwise} \end{cases}$	R
Łukasiewicz I_{Luk}	$\min(1 - x + y, 1)$	S, R

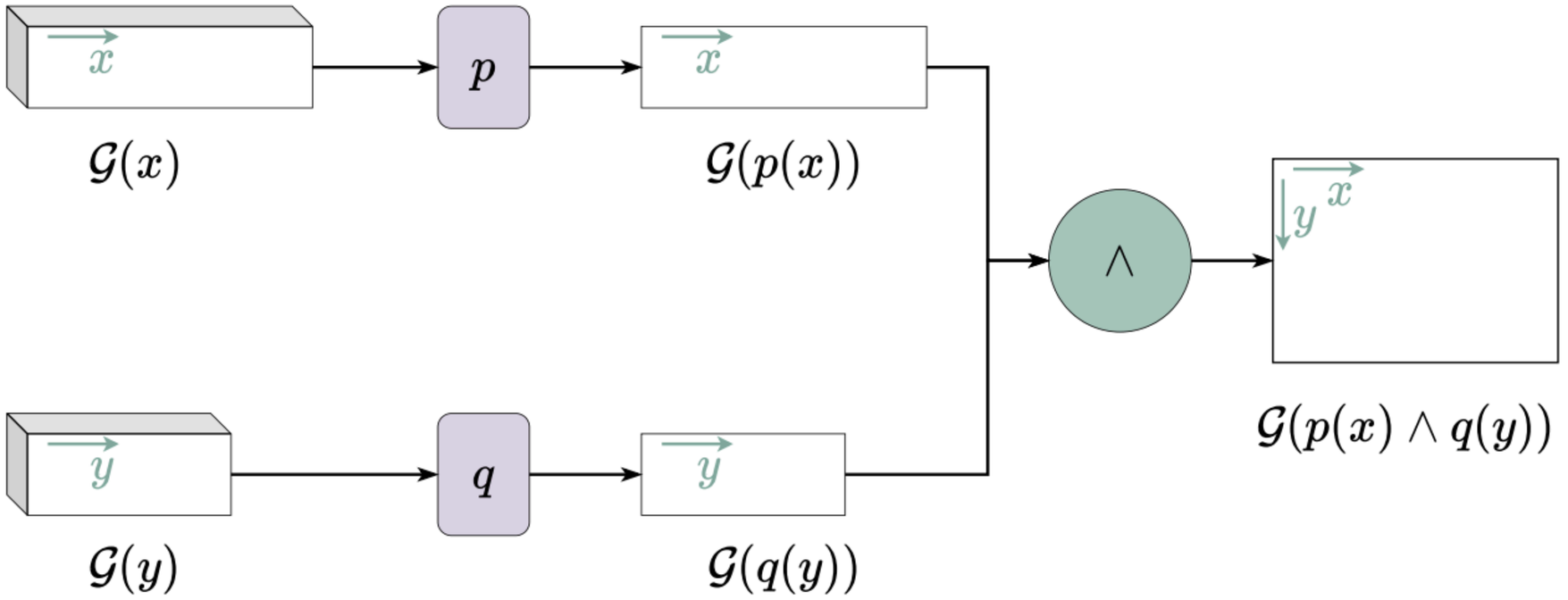
Example $a = 0.6, \; c = 0.4$

Gödel (R)	Reichenbach (S)	Łukasiewicz
$a \rightarrow_R c = 0.4$	$a \rightarrow_S c = 0.64$	$a \rightarrow c = 0.8$

$a = 0.7 \; b = 0.8$

$$\min(a, b) = 0.7$$
$$a \cdot b = 0.56$$
$$\max\{a + b - 1, 0\} = 0.5$$

Real Logic: Conjunction



Source: Badreddine, S., Garcez, A. D. A., Serafini, L., & Spranger, M. (2022). Logic tensor networks. Artif. Intelli., 303, 103649.

Real Logic: Language

Quantifiers are defined using aggregators (symmetric and continuous operators)

- **Existential** (∃): Generalization of existential quantification in FOL

- **Universal** (∀): Generalization of universal quantification in FOL

Real Logic: Common Fuzzy Operators

Some possible **aggregators** for fuzzy **universal** quantification:

$$A_{T_M}(x_1, \ldots, x_n) = \min(x_1, \ldots, x_n) \qquad \text{(Minimum)}$$

$$A_{T_P}(x_1, \ldots, x_n) = \prod_{i=1}^{n} x_i \qquad \text{(Product)}$$

$$A_{T_L}(x_1, \ldots, x_n) = \max(\sum_{i=1}^{n} x_i - n + 1, 0) \qquad \text{(Lukasiewicz)}$$

Some possible **aggregators** for fuzzy **existential** quantification:

$$A_{S_M}(x_1, \ldots, x_n) = \max(x_1, \ldots, x_n) \qquad \text{(Maximum)}$$

$$A_{S_P}(x_1, \ldots, x_n) = 1 - \prod_{i=1}^{n} (1 - x_i) \qquad \text{(Probabilistic Sum)}$$

$$A_{S_L}(x_1, \ldots, x_n) = \min(\sum_{i=1}^{n} x_i, 1) \qquad \text{(Lukasiewicz)}$$

Smooth aggregators improve gradient propagation at the cost of losing exact Boolean semantics for finite p, which are recovered in the limit as $p \to \infty$.

Real Logic: Smoothed Aggregators

$$\exists : A_{pM}(a_1, \ldots, a_n) = \left(\frac{1}{n} \sum_{i=1}^{n} a_i^p \right)^{\frac{1}{p}} \qquad p \geqslant 1$$

$$\forall : A_{pME}(a_1, \ldots, a_n) = 1 - \left(\frac{1}{n} \sum_{i=1}^{n} (1 - a_i)^p \right)^{\frac{1}{p}} \qquad p \geqslant 1$$

Designed to avoid single passing and vanishing gradient issues.

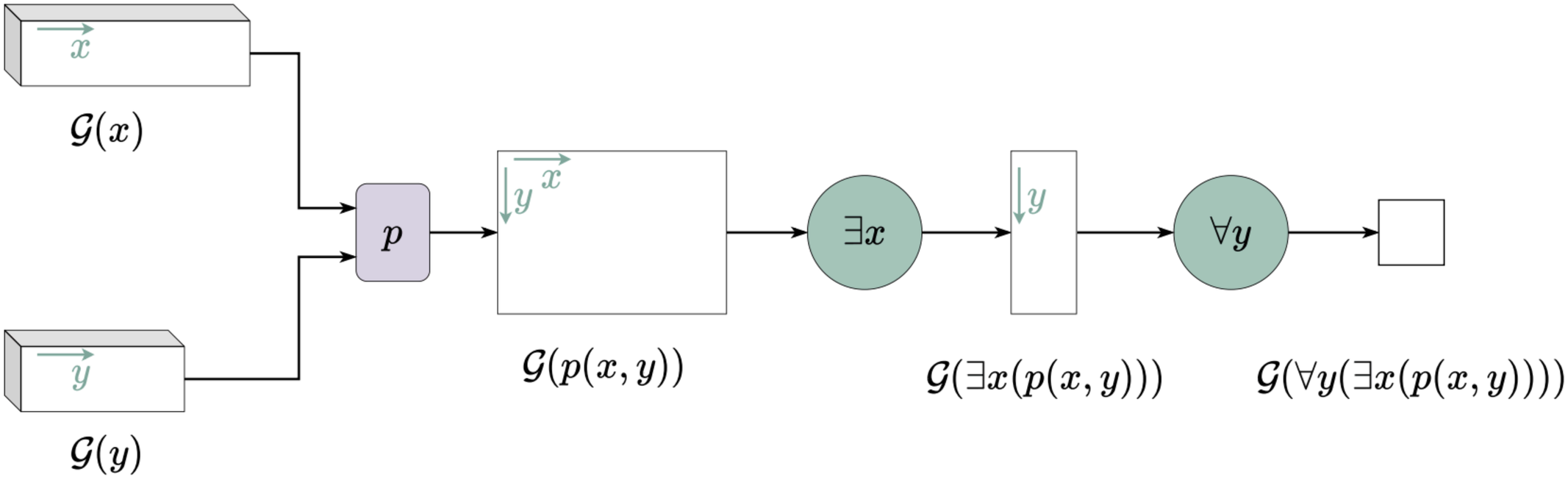
$p = 1 \to A_{pM}$ Mean

$p = 2 \to A_{pM}$ RMSE

$p = 1 \to A_{pME}$ is 1-mean

$p = 2 \to A_{pME}$ is 1-RMSE

Real Logic: Quantification



Note: Depending on the properties that the aggregation operators enjoy, the semantics may or may not adequately generalize that of FOL.

(For instance, commutativity of quantifiers may not be guaranteed.)

$$\forall x \forall y\, P(x,y) \equiv \forall y \forall x\, P(x,y)$$
$$\exists x \exists y\, P(x,y) \equiv \exists y \exists x\, P(x,y)$$
$$\forall x\, P(x) \equiv \neg \exists x \neg P(x)$$

Source: Badreddine, S., Garcez, A. D. A., Serafini, L., & Spranger, M. (2022). Logic tensor networks. Artif. Intelli., 303, 103649.

Real Logic: example of grounding

$friend(Mary, John)$

Individuals are grounded with real features (e.g. vectors, matrices, ...).

e.g. $\mathcal{G}(Mary) = [6.2, 1.5, \ldots] \in \mathbb{R}^m$

Predicates are grounded with operations (e.g. neural networks, ...) that project in the interval [0,1].
The output denotes a *satisfaction level*.

e.g. $\mathcal{G}(friend) : \mathbb{R}^m \times \mathbb{R}^m \longrightarrow [0, 1]$

$\forall x(friend(John, x) \rightarrow friend(Mary, x))$

Connectives ($\wedge, \vee, \rightarrow, \neg$) are interpreted using fuzzy semantics

e.g. $0.7 \wedge_{\text{prod}} 0.2 = 0.7 \cdot 0.2 = 0.14$

Variables are grounded as a list of n individuals

e.g. $x \in \mathbb{R}^{n \times m}$

Quantifiers ($\forall, \exists$) are interpreted as aggregators

e.g. $\forall_{\text{mean}}(0.7, 0.2, \ldots) = \frac{1}{n}(0.7 + 0.2 + \ldots)$

- $friend(Mary, John)$ is an atomic formula (**predicate**)
- $\forall x \ (\ friend(John, x) \Rightarrow friend(Mary, x) \ )$ is a closed **formula** (all the variables are quantified)
- $\mathcal{G}$ is the **grounding**, i.e. the function which maps the logical domain into the real domain.

Real Logic: Language

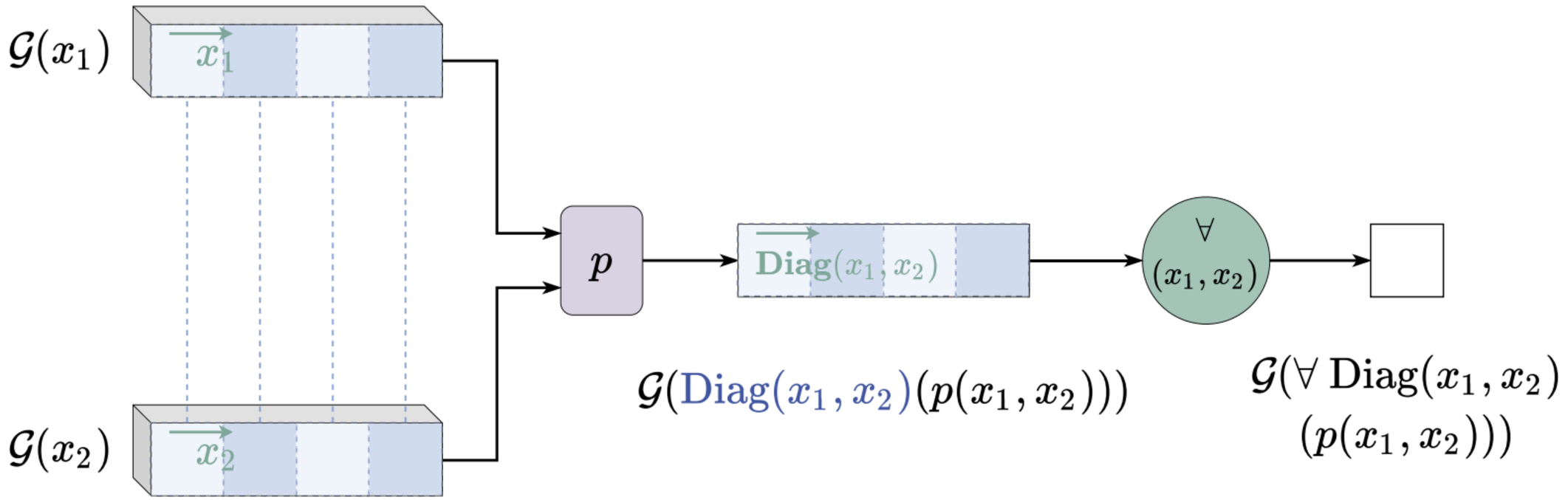
Diagonal quantification:

- $Diag(x_1, \ldots, x_h)$ quantifies over specific tuples s.t. the i-th tuple contains the i-th instance of each of the variables in the argument of $Diag$.
- Assumes that all variables in the argument are grounded onto sequences with the same number of instances.

Guarded quantification:

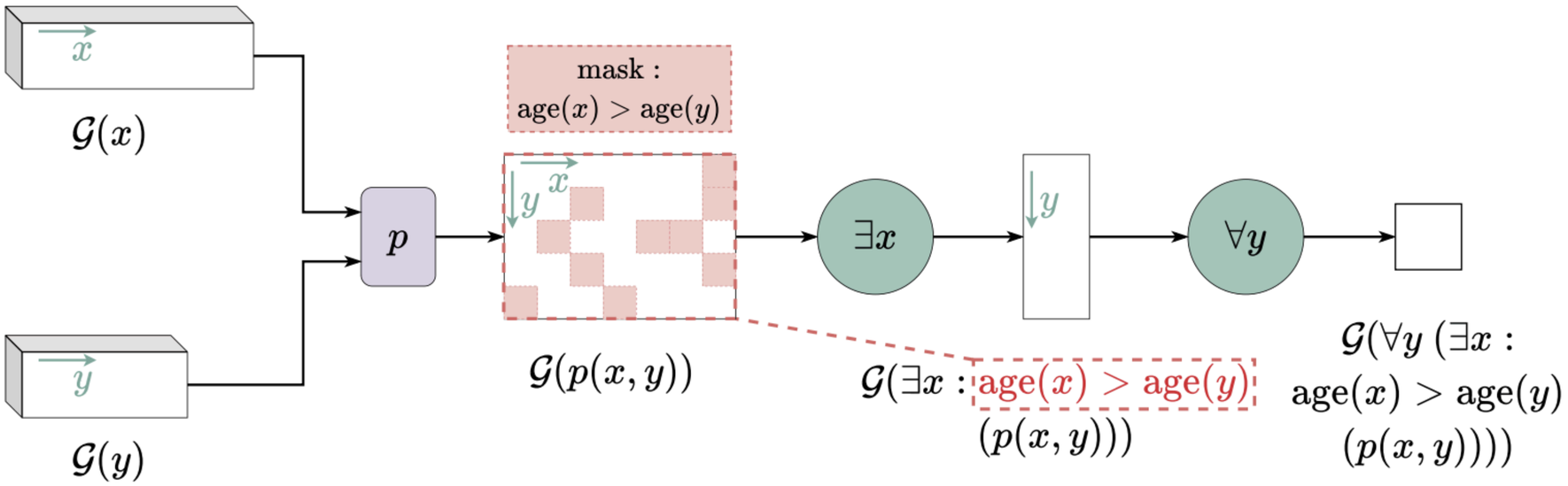
- Quantifies over a subset of variables that satisfy a condition m (mask).

Real Logic: Diagonal Quantification



Source: Badreddine, S., Garcez, A. D. A., Serafini, L., & Spranger, M. (2022). Logic tensor networks. Artif. Intelli., 303, 103649.

Real Logic: Guarded Quantification



$$\mathcal{G}(Q\ x_1,\ldots,x_h : m(x_1,\ldots,x_n)(\phi))_{i_{h+1},\ldots,i_n} \overset{\text{def}}{=} \operatorname*{Agg}_{\substack{(Q)\\ i_1=1,\ldots,|\mathcal{G}(x_1)|\\ \vdots\\ i_h=1,\ldots,|\mathcal{G}(x_h)| \text{ s.t.}\\ \mathcal{G}(m)(\mathcal{G}(x_1)_{i_1},\ldots,\mathcal{G}(x_n)_{i_n})}} \mathcal{G}(\phi)_{i_1,\ldots,i_h,i_{h+1},\ldots,i_n}$$

Source: Badreddine, S., Garcez, A. D. A., Serafini, L., & Spranger, M. (2022). Logic tensor networks. Artif. Intelli., 303, 103649.

From Real Logic to LTNs

LTN computational graph

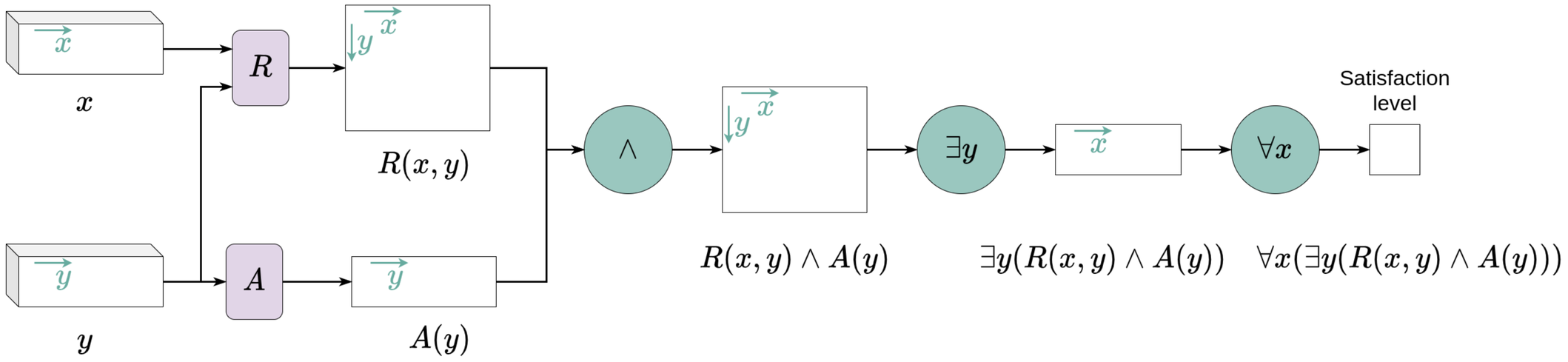
LTN converts **Real Logic formulas into a computational graph**.

Such formulas can express complex queries about the data, prior knowledge to satisfy during learning, statements to prove, etc.

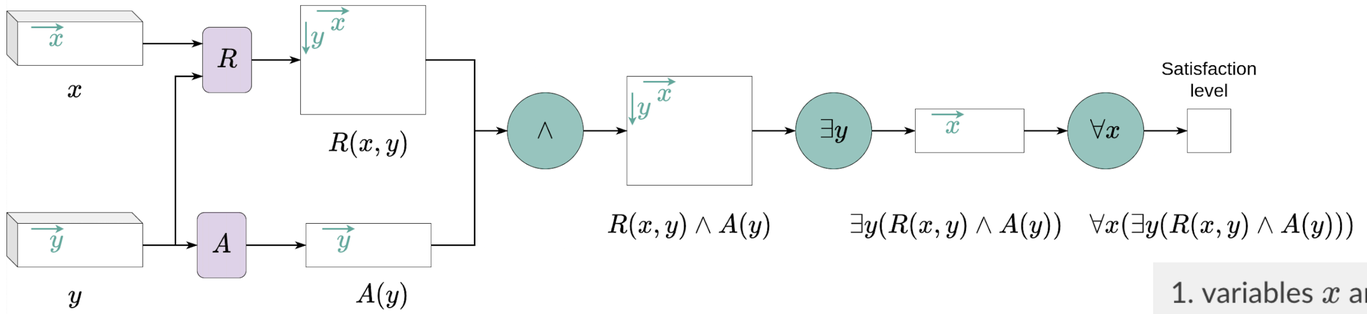
From Real Logic to Tensor Networks: example

Real Logic formula: $\forall x \exists y \ (friend(x, y) \wedge italian(y))$ stating that everybody has a friend that is Italian

e.g. if R denotes the predicate for $friends$, and A denotes the predicate for $Italian$,
the following computational graph translates the English sentence "everybody has a friend that is Italian".



Source: Badreddine, S., Garcez, A. D. A., Serafini, L., & Spranger, M. (2022). Logic tensor networks. Artif. Intelli., 303, 103649.



In practice, LTN converts Real Logic formulas (e.g., $\forall x \exists y \ (friend(x, y) \wedge italian(y))$, which states that everybody has a friend that is Italian) into PyTorch computational graphs. Such formulas can express complex queries about the data, prior knowledge to satisfy during learning, statements to prove, etc.

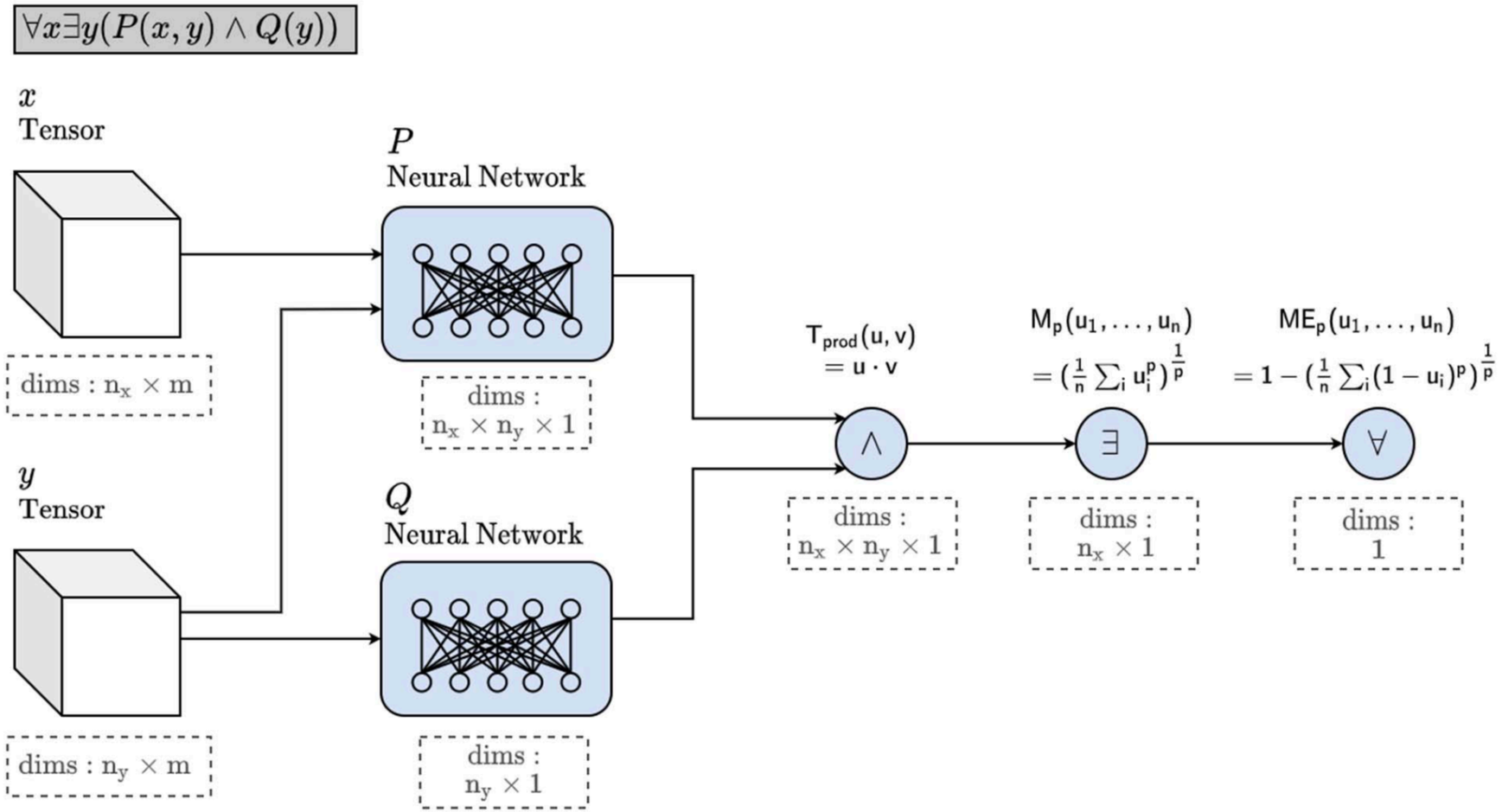
LTN converts such formulas into PyTorch computational graphs.

Let us assume we have 6 people which are denoted using 4 real-valued features.

The figure above illustrates the following:

1. variables x and y are two sequences of our 6 people, namely $\mathcal{G}(x) = \mathcal{G}(y) = \mathbb{R}^{6\times4}$. In the figure, the features of the people are depicted in the depth dimension;
2. R is a binary predicate which takes as input two variables, x and y, and returns a matrix $\mathbf{M} \in [0., 1.]^{6\times6}$, which is the result of the evaluation of predicate R on all pairs of people. In the figure, the first dimension of the matrix is associated with variable x, while the second dimension with variable y. For example, the truth value in position [0, 2] is the result of the evaluation of predicate R for the first individual of x and third individual of y;
3. A is a unary predicate which takes as input one variable, y, and returns a vector $\mathbf{v} \in [0., 1.]^6$, which is the result of the evaluation of predicate A on each individual of y. In the figure, there is one single dimension, which is associated with variable y;
4. $\wedge$ is a binary fuzzy operator which generalizes the logical conjunction. The operator is applied element-wise to $\mathbf{M}$ and $\mathbf{v}$, and returns a new matrix $\mathbf{N} \in [0., 1.]^{6\times6}$, which is the fuzzy logical conjunction of $\mathbf{M}$ and $\mathbf{v}$. If the shapes of the operands (inputs) are different, like in this case, broadcasting is applied. If you are interested, the PyTorch implementation of common fuzzy operators is available at `ltn.fuzzy_ops`;
5. $\exists y$ is a fuzzy aggregator which generalizes the existantial quantification. The aggregator takes as input matrix $\mathbf{N}$ and aggregates along the dimension associated with variable y, resulting in a vector $\mathbf{u} \in [0., 1.]^6$, which containes the result of the aggregation;
6. $\forall x$ is a fuzzy aggregator which generalizes the universal quantification. The aggregator takes as input vector $\mathbf{u}$ and aggregates along the dimension associated with variable x, resulting in a scalar $c \in [0., 1.]$, which containes the result of the aggregation. This result is also the final result of the initial formula $\forall x \exists y(friend(x, y) \wedge italian(y))$, which can be interpreted as a satisfaction level for the formula.

From Real Logic to Tensor Networks



Source: https://github.com/logictensornetworks

Differentiable Fuzzy Logic

In presenting Real Logic and LTN, we did not give much thought to the role of fuzzy operators in the main tasks.

Gradient descent requires that operators be differentiable so that it can smoothly traverse the universe of values.

Three types of **gradient problems** commonly arise:

1. **Single-Passing:** The derivatives of some operators are non-null for only one argument. The gradients propagate to only one input at a time.
2. **Vanishing Gradients:** Gradients vanish on some part of the domain. Learning does not update inputs that are in the vanishing domain.
3. **Exploding Gradients:** Large error gradients accumulate and result in unstable updates.

Problems: Single-Passing Gradients

Some operators have gradients propagating to only one input at a time, meaning that all other inputs will not benefit from learning at this step.

e.g. in $\min(u_1, \ldots, u_n)$.

```
xs = tf.constant([1.,1.,1.,0.5,0.3,0.2,0.2,0.1])

with tf.GradientTape() as tape:
    tape.watch(xs)
    y = forall_min(xs)
res = y.numpy()
gradients = tape.gradient(y,xs).numpy()
print(res)
print(gradients)
```

```
0.1
[0. 0. 0. 0. 0. 0. 0. 1.]
```

Problems: Vanishing Gradients

Some operators have vanishing gradients on some part of their domains.

e.g. in $u \wedge_{\text{luk}} v = \max(u + v - 1, 0)$, if $u + v - 1 < 0$, the gradients vanish.

```
x1 = tf.constant(0.3)
x2 = tf.constant(0.5)

with tf.GradientTape() as tape:
    tape.watch(x1)
    tape.watch(x2)
    y = and_luk(x1,x2)
res = y.numpy()
gradients = [v.numpy() for v in tape.gradient(y,[x1,x2])]
print(res)
print(gradients)
```

```
0.0
[0.0, 0.0]
```

Problems: Exploding Gradients

Some operators have exploding gradients on some part of their domains.

e.g. in $\mathbf{pME}(u_1, \ldots, u_n) = 1 - \left(\frac{1}{n} \sum_{i=1}^{n} (1 - u_i)^p \right)^{\frac{1}{p}}$, on the edge case where all inputs are 1.0.

```
xs = tf.constant([1.,1.,1.])

with tf.GradientTape() as tape:
    tape.watch(xs)
    y = forall_pME(xs,p=4)
res = y.numpy()
gradients = tape.gradient(y,xs).numpy()
print(res)
print(gradients)
```

```
1.0
[nan nan nan]
```

Problems: Binary Connectives

Gradient problems for some binary connectives. (✗) means that the problem only appears on an edge case.

	Single-Passing	Vanishing	Exploding
Goedel (minimum)			
T_M, S_M	✗		
I_{KD}	✗		
I_G	✗	✗	
Goguen (product)			
T_P, S_P		(✗)	
I_R		(✗)	
I_{KD}		✗	(✗)
Łukasiewicz			
T_L, S_L		✗	
I_{Luk}		✗	

Problems: Aggregators

Gradient problems for some aggregators. (✗) means that the problem only appears on an edge case.

	Single-Passing	Vanishing	Exploding
A_{T_M}/A_{S_M}	✗		
A_{T_P}/A_{S_P}		✗	
A_{T_L}/A_{S_L}		✗	
A_{pM}			(✗)
A_{pME}			(✗)

Stable Configuration of Operators

The following is proposed as a stable configuration:

- not: the standard negation $\neg u = 1 - u$,
- and: the product t-norm $u \wedge v = uv$,
- or: the product t-conorm (probabilistic sum) $u \vee v = u + v - uv$,
- implication: the Reichenbach implication $u \to v = 1 - u + uv$,
- existential quantification ("exists"): the generalized mean (p-mean)

$$\mathrm{pM}(u_1, \ldots, u_n) = \left(\frac{1}{n} \sum_{i=1}^{n} u_i^p \right)^{\frac{1}{p}} \qquad p \geq 1,$$

- universal quantification ("for all"): the generalized mean of "the deviations w.r.t. the truth" (p-mean error)

$$\mathrm{pME}(u_1, \ldots, u_n) = 1 - \left(\frac{1}{n} \sum_{i=1}^{n} (1 - u_i)^p \right)^{\frac{1}{p}} \qquad p \geq 1.$$

Stable Configuration of Operators

Some caveats:

- The product t-norm has vanishing gradients on the edge case $u = v = 0$.
- The product t-conorm has vanishing gradients on the edge case $u = v = 1$.
- The Reichenbach implication has vanishing gradients on the edge case $u = 0, v = 1$.
- p-mean has exploding gradients on the edge case $u_1 = \cdots = u_n = 0$.
- p-mean error has exploding gradients on the edge case $u_1 = \cdots = u_n = 1$.

Stable Configuration of Operators

These issues happen on edge cases and can be fixed using the following "**trick**":

- if the edge case happens when an input u is 0, we modify every input with

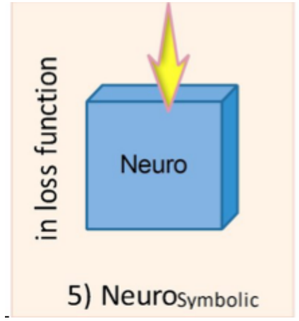
$$u' = (1 - \epsilon)u + \epsilon$$

- if the edge case happens when an input u is 1, we modify every input with

$$u' = (1 - \epsilon)u$$

where ϵ is a small positive value (e.g., 1e–5).

LTN within the Kautz Taxonomy



According to Kautz, LTN fall within the **Neuro_{Symbolic}** category **(type 5)**:

- These architectures transform symbolic rules into templates for structures within the neural network.

- How is this different from category 1 (Symbolic Neuro symbolic)?
  - Only the objects (individuals) are tensorized
  - It's the standard way in which deep learning approaches work: after processing input tensors, output tensors are produced and transformed back into symbols.
  - LTN tensorizes all logical components: objects, predicates, functions, and formulas.

LTN: Tasks

One can define the tasks of:

- **Learning:** The task of making generalizations from specific observations obtained from data (often called inductive inference)

- **Reasoning:** The task of deriving what knowledge follows from the facts that are currently known (often called deductive inference).

- **Query answering (QA):** The task of evaluating the truth value of a certain logical expression (query), or finding the set of objects in the data that evaluate a certain expression to true.

LTN: Tasks

3 basic tasks:

A. **Learning** (inductive inference): task of generalizing from specific observations (i.e. data)

B. **Query Answering**: evaluation of the truth of a given logical expression (a query). The task can also refer to the search for a set of objecs that satisfy a given query

C. **Reasoning** (deductive inference): deriving what follows from a set of facts

Let $\mathscr{T} = \langle \mathscr{K}, \mathscr{G}(\,\cdot\,|\theta), \Theta \rangle$ denote Real Logic **theories**:

$\mathscr{K}$: set of closed first order logic formulas

$\mathscr{G}(\,\cdot\,|\theta)$: parametric grounding for all symbols (θ is the set of parameter values)

Θ: set of possible parameter values, aka hypothesis space

Theories with fixed parameter values are called *grounded theories* and denoted with $\langle \mathscr{K}, \mathscr{G}_\theta \rangle$.

A. LTN Satisfaction

LEARNING A THEORY

- Learning and reasoning in a Real Logic theory are both associated with searching for and applying the set of values of parameters θ from the hypothesis space Θ that maximize the satisfaction of the formulas in the knowledge base.
- To define this optimization problem, we aggregate the truth values of all the formulas in KB $\mathcal{K}$ by selecting a formula aggregating operator:

$$SatAgg : [0,1]^* \rightarrow [0,1]$$

- Learning is the process of searching for the set of parameter values θ^* that maximize the satisfiability of the RL theory w.r.t. a given aggregator:

$$\theta^* = \arg\max_{\theta \in \Theta} SatAgg_{\phi \in \mathcal{K}} \mathcal{G}_\theta(\phi)$$

A. LTN Satisfaction

With this general formulation, one can learn the grounding of constants, functions, and predicates:

- Learning grounding of **constants** corresponds to learning of **embeddings**.
- Learning grounding of **functions** corresponds to learning **generative models** or a **regression** task.
- Learning of the grounding of **predicates** corresponds to a **classification** task.

B. LTN Querying

When provided with a ground truth theory, query answering (QA) is the process of **checking** if a certain condition (**fact**) holds (is true, better *how much it is true*), which is expressed via a formula called a query.

Various types of queries:

- **Truth queries**: asks for the truth value of a formula in the language, which can be closed (i.e. with no variables, which yields a scalar) or not (i.e. that has n free variables, which yield a tensor of order n)
- **Value queries**: analogous to truth queries, except that they are evaluated over terms rather than formulas. The aswer to such queries is the tensor that corresponds to the grounding of the term. If the query has free variables then it yields a tensor of tensors as its answers.
- **Generalization truth queries:** What is the truth value associated to a formula evaluated over unseen data?
- **Generalization value queries:** Analogous

C. LTN Reasoning

- **Reasoning** is the task of verifying if a formula is a logical consequence of a set of formulas.

- A formula ϕ is a fuzzy **logical consequence** of a finite set of formulas Γ iff every model of Γ is a model of ϕ.

- In Real Logic, this is **generalized** by defining an interval $[q,1]$ with $0.5 < q < 1$ and assuming that a formula is true iff its truth-value is in the interval $[q,1]$.

> **Definition 6.** A closed formula ϕ is a logical consequence of a knowledge-base $(\mathcal{K}, \mathcal{G}(\cdot \mid \boldsymbol{\theta}), \boldsymbol{\Theta})$, in symbols $(\mathcal{K}, \mathcal{G}(\cdot \mid \boldsymbol{\theta}), \boldsymbol{\Theta}) \models_q \phi$, if, for every grounded theory $\langle \mathcal{K}, \mathcal{G}_{\boldsymbol{\theta}} \rangle$, if $\mathrm{SatAgg}(\mathcal{K}, \mathcal{G}_{\boldsymbol{\theta}}) \geqslant q$ then $\mathcal{G}_{\boldsymbol{\theta}}(\phi) \geqslant q$.

- Unfortunately, this definition is only useful in theory since it requires inspecting potentially **infinite** sets of groundings (i.e. requires querying the truth value of ϕ for a potentially infinite set of groundings).

LTN Approximate Reasoning



This approach is somewhat naive. Even if we run the optimization multiple times with multiple parameter initializations, the obtained groundings may not be representative of other optimal or close-to-optimal groundings.

Option 1: Querying after learning

- Consider only the grounded theories that maximally satisfy the given theory.
- A (likely incomplete) set of such grounded theories can be found via **multiple optimization runs**
- This is a kind of **brave reasoning** → **reasoning by optimization**.

Reasoning Option 1 (Querying after learning). This is approximate logical inference by considering only the grounded theories that maximally satisfy $(\mathcal{K}, \mathcal{G}(\cdot \mid \boldsymbol{\theta}), \boldsymbol{\Theta})$. We therefore define that ϕ is a *brave logical consequence* of a Real Logic knowledge-base $(\mathcal{K}, \mathcal{G}(\cdot \mid \boldsymbol{\theta}), \boldsymbol{\Theta})$ if $\mathcal{G}_{\boldsymbol{\theta}*}(\phi) \geqslant q$ for all the $\boldsymbol{\theta}^*$ such that:

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \operatorname{SatAgg}(\mathcal{K}, \mathcal{G}_{\boldsymbol{\theta}}) \qquad \text{and} \qquad \operatorname{SatAgg}(\mathcal{K}, \mathcal{G}_{\boldsymbol{\theta}*}) \geqslant q$$

The objective is to find all $\boldsymbol{\theta}^*$ that optimally satisfy the knowledge base and to measure if they also satisfy ϕ. One can search for such $\boldsymbol{\theta}^*$ by running multiple optimizations

$$\mathcal{G}(\mathcal{K}) = SatAgg_{\phi \in \mathcal{K}}(\mathcal{K}, \mathcal{G})$$

LTN Approximate Reasoning

Option 2: Proof by refutation

- Search for a **counterexample** to the consequence.
- If no such example is found, the consequence is **assumed** to hold.

Reasoning Option 2 (Proof by Refutation). Here, we reason by refutation and search for a counter-example to the logical consequence by introducing an alternative search objective. Normally, according to Definition 6, one tries to verify that:[11]

$$\text{for all } \theta \in \Theta, \text{ if } \mathcal{G}_\theta(\mathcal{K}) \geqslant q \text{ then } \mathcal{G}_\theta(\phi) \geqslant q. \tag{21}$$

Instead, we solve the dual problem:

$$\text{there exists } \theta \in \Theta \text{ such that } \mathcal{G}_\theta(\mathcal{K}) \geqslant q \text{ and } \mathcal{G}_\theta(\phi) < q. \tag{22}$$

LTN Approximate Reasoning

Option 2: Proof by refutation

A search for such parameters θ (the counterexample) can be performed by minimizing $\mathscr{G}_\theta(\phi)$ while imposing a constraint that seeks to invalidate results where $\mathscr{G}_\theta(\mathscr{K}) < q$. We therefore define:

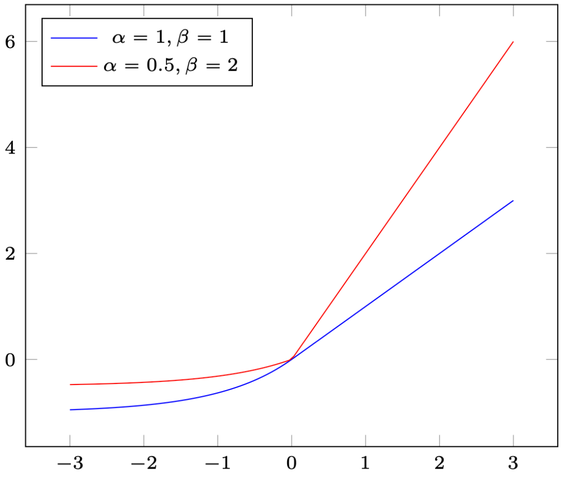
$$\text{penalty}(\mathcal{G}_\theta, q) = \begin{cases} c \text{ if } \mathcal{G}_\theta(\mathcal{K}) < q, \\ 0 \text{ otherwise,} \end{cases} \qquad \text{where } c > 1.$$ [12]

Given $\mathcal{G}^*$ such that:

$$\mathcal{G}^* = \underset{\mathcal{G}_\theta}{\text{argmin}}(\mathcal{G}_\theta(\phi) + \text{penalty}(\mathcal{G}_\theta, q)) \tag{23}$$

- If $\mathcal{G}^*(\mathcal{K}) < q$: Then for all $\mathcal{G}_\theta$, $\mathcal{G}_\theta(\mathcal{K}) < q$ and therefore $(\mathcal{K}, \mathcal{G}(\,\cdot \mid \boldsymbol{\theta}), \boldsymbol{\Theta}) \models_q \phi$.
- If $\mathcal{G}^*(\mathcal{K}) \geqslant q$ and $\mathcal{G}^*(\phi) \geqslant q$: Then for all $\mathcal{G}_\theta$ with $\mathcal{G}_\theta(\mathcal{K}) \geqslant q$, we have that $\mathcal{G}_\theta(\phi) \geqslant \mathcal{G}^*(\phi) \geqslant q$ and therefore $(\mathcal{K}, \mathcal{G}(\,\cdot \mid \boldsymbol{\theta}), \boldsymbol{\Theta}) \models_q \phi$.
- If $\mathcal{G}^*(\mathcal{K}) \geqslant q$ and $\mathcal{G}^*(\phi) < q$: Then $(\mathcal{K}, \mathcal{G}(\,\cdot \mid \boldsymbol{\theta}), \boldsymbol{\Theta}) \not\models_q \phi$.

LTN Approximate Reasoning



Option 2: Proof by refutation

Equation (23) cannot be used as an objective function for gradient-descent due to null derivatives. Therefore, we approximate the penalty function with the **soft constraint**:

$$\texttt{elu}(\alpha, \beta(q - \mathcal{G}_\theta(\mathcal{K}))) = \begin{cases} \beta(q - \mathcal{G}_\theta(\mathcal{K})) & \text{if } \mathcal{G}_\theta(\mathcal{K}) \leqslant q, \\ \alpha(e^{q - \mathcal{G}_\theta(\mathcal{K})} - 1) & \text{otherwise,} \end{cases}$$

where $\alpha \geq 0$ and $\beta \geq 0$ are hyper-parameters.

When $\mathscr{G}_\theta(\mathscr{K}) < q$, the penalty is linear in $q - \mathscr{G}_\theta(\mathscr{K})$ with a slope of β.

When $\mathscr{G}_\theta(\mathscr{K}) \geq q$, the penalty is a negative exponential that converges to $-\alpha$. Setting α low but non-zero seeks to ensure that the gradients do not vanish when the penalty should not apply (when the knowledge-base is satisfied).

Approx. Reasoning: Example $A \vee B \models_q A$?

$\mathcal{G}(A) = a$, $\mathcal{G}(B) = b$, $\theta = \{a, b\}$. At initialization $a = b = 0$.

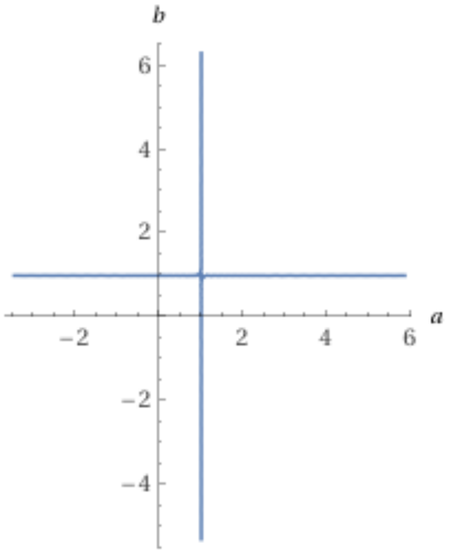
$$\mathcal{G}_\theta(\mathcal{K}) = \mathcal{G}_\theta(A \vee B) = a + b - ab$$

Example showing that reasoning by refutation is the preferable option when we seek to find out whether $A \vee B \models_q A$. That is, given the threshold $q = 0.95$, does every $\mathcal{G}_\theta$ such that $\mathcal{G}_\theta(\mathcal{K}) \geq q$ verify $\mathcal{G}_\theta(\phi) \geq q$.

There are infinite global optima maximizing the satisfiability of the theory.

For instance:

$$\mathcal{G}_1\colon\ \mathcal{G}_1(A) = 1, \mathcal{G}_1(B) = 1, \mathcal{G}_1(\mathcal{K}) = 1,$$
$$\mathcal{G}_2\colon\ \mathcal{G}_2(A) = 1, \mathcal{G}_2(B) = 0, \mathcal{G}_2(\mathcal{K}) = 1,$$
$$\mathcal{G}_3\colon\ \mathcal{G}_3(A) = 0, \mathcal{G}_3(B) = 1, \mathcal{G}_3(\mathcal{K}) = 1.$$



a+b-ab=1

Approx. Reasoning: Example $A \vee B \models_q A$?

$A \vee B \models_q A$? Immediately, one can notice that this is not the case. For instance, the grounding $\mathcal{G}_3$ is a counter-example.

If one simply reasons by querying multiple groundings after learning with the usual objective $\text{argmax}_{(\mathcal{G}_\theta)} \mathcal{G}_\theta(\mathcal{K})$, the results will all converge to $\mathcal{G}_1$: $\frac{\partial \mathcal{G}_\theta(\mathcal{K})}{\partial a} = 1 - b$ and $\frac{\partial \mathcal{G}_\theta(\mathcal{K})}{\partial b} = 1 - a$. Every run of the optimizer will increase a and b simultaneously until they reach the optimum $a = b = 1$. Because the grid search always converges to the same point, no counter-example is found and the logical consequence is mistakenly assumed true. This is illustrated in Fig. 22

Reasoning by refutation, however, the objective function has an incentive to find a counter-example with $\neg A$, as illustrated in Figure 23. LTN converges to the optimum $\mathcal{G}_3$, which refutes the logical consequence.

Approx. Reasoning: Option 1 vs 2 for $A \vee B \models A$?

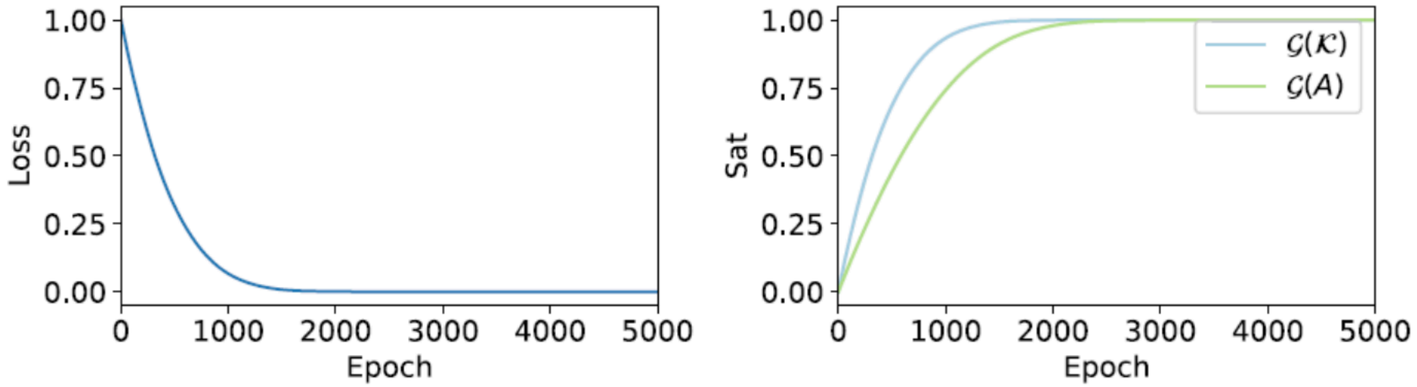


Fig. 22. Querying after learning: 10 runs of the optimizer with objective $\mathcal{G}^* = \text{argmax}_{\mathcal{G}_\theta}(\mathcal{G}_\theta(\mathcal{K}))$. All runs converge to the optimum $\mathcal{G}_1$; the grid search misses the counter-example.

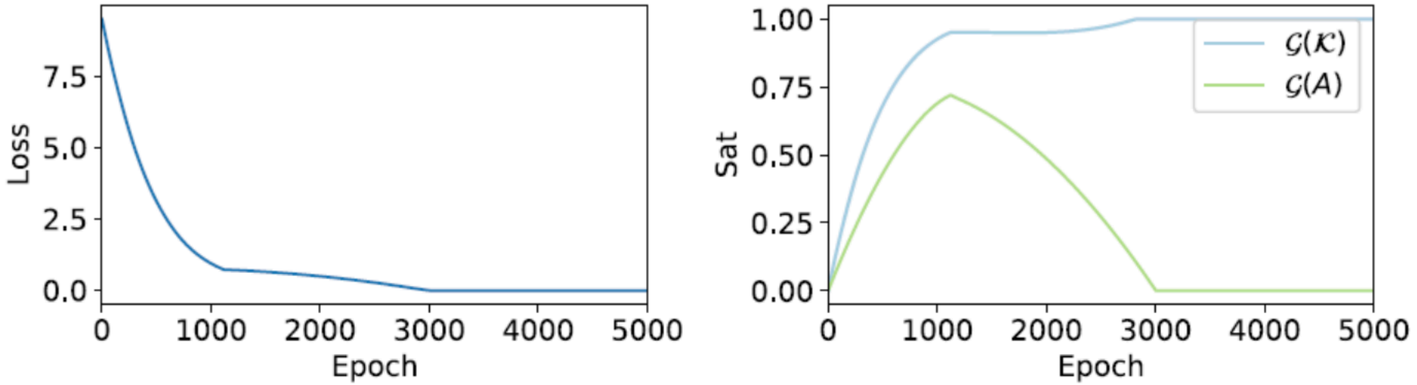


Fig. 23. Reasoning by refutation: one run of the optimizer with objective $\mathcal{G}^* = \text{argmin}_{\mathcal{G}_\theta}(\mathcal{G}_\theta(\phi) + \texttt{elu}(\alpha, \beta(q - \mathcal{G}_\theta(\mathcal{K}))), q = 0.95, \alpha = 0.05, \beta = 10$. In the first training epochs, the directed search prioritizes the satisfaction of the knowledge base. Then, the minimization of $\mathcal{G}_\theta(\phi)$ starts to weigh in more and the search focuses on finding a counter-example. Eventually, the run converges to the optimum $\mathcal{G}_3$, which refutes the logical consequence.

Hands-On

Logic Tensor Network

Real Logic - hands on

- Install LTN: `pip install LTNtorchv`
- Constants, predicates, functions, variables:

  https://nbviewer.org/github/tommasocarraro/LTNtorch/blob/main/tutorials/1-grounding_non_logical_symbols.ipynb

- Connectives and quantifiers:

  https://nbviewer.org/github/tommasocarraro/LTNtorch/blob/main/tutorials/2-grounding_connectives.ipynb

Learning in LTN - hands on

Using satisfiability of LTN formulas as a training objective

https://nbviewer.org/github/tommasocarraro/LTNtorch/blob/main/tutorials/3-knowledgebase-and-learning.ipynb

Multi-class multi-label classification

https://github.com/tommasocarraro/LTNtorch/blob/main/examples/3-multi_class_multi_label_classification.ipynb

Tutorials

The examples covered are the following:

1. Binary classification: illustrates, in the simplest setting, how to ground a binary classifier as a predicate in LTN;
2. Multi-class single-label classification: illustrate how to ground predicates that can classify samples in several mutually-exclusive classes;
3. Multi-class multi-label classification: illustrate how to ground predicates that can classify samples in several classes which are not mutually-exclusive;
4. Semi-supervised pattern recognition: showcases the power of LTN in dealing with semi-supervised learning tasks;
5. Regression: illustrates how to ground a regressor as a function symbol in LTN;
6. Clustering: illustrates how LTN can solve a unsupervised tasks using first-order logical constraints;
7. Learning embeddings with LTN: illustrates how LTN can learn embeddings using learnable logical constants.

The examples are presented using jupyter notebooks.

Coming next…

NeurASP